DATA STRUCTURES · CORE CODE

数据结构 14 段核心代码

分类整理 · 打印友好 · 背诵手册

按知识模块分为六章，覆盖 14 段核心代码；逐段给出作用、复杂度、记忆骨架、逐行注释与易错点，适合打印后遮码默写。

四色标注对照

红 · 必背核心语句　蓝 · 关键结构 / 术语　绿 · 易错边界条件　橙 · 复杂度 / 结论

整理说明

- 14 段代码按“线性表 → 串 → 树 → 图 → 查找 → 排序”归入六章，BFS 另分邻接表、邻接矩阵两页对照。
- 每段配“一句话作用 + 复杂度 + 骨架 + 背诵要点 + 默写自检”，适合打印后遮住代码自测。
- 四色只标重点，不铺满版面；黑白打印时靠加粗与底纹深浅仍可区分。
- 文末附 5 张速查表、高频易错清单、三轮默写路线与默写自检表。

第一章	线性表 · 单链表的建立（2 段）
第二章	串 · 模式匹配（2 段）
第三章	树与二叉树 · 四种遍历（4 段）
第四章	图 · BFS / DFS / 拓扑排序（4 段）
第五章	查找 · 折半查找（1 段）
第六章	排序 · 快速排序 / 归并排序（2 段）
附　录	速查表 · 易错清单 · 默写自检表

目　录

编号为复习顺序，按章排列；页边留白可直接写批注。

第一章　线性表 · 单链表的建立

01　头插法建立单链表　3

02　尾插法建立单链表　4

第二章　串 · 模式匹配

03　求模式串 T 的 next 数组　5

04　KMP 算法　6

第三章　树与二叉树 · 四种遍历

05　二叉树的先序遍历　7

06　二叉树的中序遍历　8

07　二叉树的后序遍历　9

08　二叉树的层序遍历　10

第四章　图 · 遍历与拓扑排序

09A　BFS（初始化 + 邻接表）　11

09B　BFS（邻接矩阵）　12

10　DFS 深度优先搜索　13

11　拓扑排序　14

第五章　查找

12　折半（二分）查找　15

第六章　排序

13　快速排序　16

14　二路归并排序　17

附　录

A　速查表一：四种遍历 / BFS 与 DFS　18

B　速查表二：排序 / 查找 / 链表　18

C　高频易错清单 · 三轮默写路线 · 自检表　18

一　线性表 · 单链表的建立

链表是“指针游戏”：先接后断，尾插记得收尾。

01　头插法建立单链表

从空表开始，每次把新结点插到头结点之后。输入序列与链表顺序相反。

时间 O(n)　空间 O(n)　头指针 L

> **骨架**　建头结点 → L->next 置空 → 读 x → 循环：新建 s、填值、插到表头 → 返回 L

```
LinkList List_HeadInsert(LinkList &L)       // 头插法建立单链表
{
    LNode *s;
    int x;
    L = (LNode *)malloc(sizeof(LNode));     // 创建头结点
    L->next = NULL;                         // 初始为空链表
    scanf("%d", &x);                        // 输入结点的值
    while (x != 9999)                       // 输入 9999 表示结束
    {
        s = (LNode *)malloc(sizeof(LNode));// 创建新结点
        s->data = x;
        s->next = L->next;
        L->next = s;                        // 将新结点插入表中，L 为头指针
        scanf("%d", &x);
    }
    return L;
}
```

背诵要点

- 核心　头插法的结果与输入顺序 **相反**，常用于链表逆置。
- 易错　插入两句顺序固定：先 `s->next = L->next;`，再 `L->next = s;`。
- 易错　9999 只是约定的结束标志；循环体内要再读一次 x。

默写自检：☐ 能写出框架　☐ 能写对边界　☐ 能说出复杂度

02　尾插法建立单链表

新结点始终接在表尾，输入顺序与链表顺序一致。

时间 O(n)　空间 O(n)　尾指针 r

> **骨架**　建头结点 → r = L → 循环：s 接在 r 之后、r 后移 → r->next 置空 → 返回 L

```
LinkList List_TailInsert(LinkList &L)      // 尾插法建立单链表
{
    int x;                                 // 设元素类型为整型
    L = (LNode *)malloc(sizeof(LNode));    // 创建头结点
    LNode *s, *r = L;                      // r 为尾指针
    L->next = NULL;                        // 初始为空链表
    scanf("%d", &x);                       // 输入结点的值
    while (x != 9999)                      // 输入 9999 表示结束
    {
        s = (LNode *)malloc(sizeof(LNode));// 创建新结点
        s->data = x;
        r->next = s;
        r = s;                             // r 指向新的尾结点
        scanf("%d", &x);
    }
    r->next = NULL;                        // 尾结点指针置空
    return L;
}
```

背诵要点

- 术语　r 是尾指针，初值 `r = L`：此时头结点同时是尾结点。
- 易错　循环结束后 `r->next = NULL` 不能漏，否则尾结点指针未置空。
- 结论　与头插法只差两点：插在哪、谁后移。

默写自检：☐ 能写出框架　☐ 能写对边界　☐ 能说出复杂度

二 串 · 模式匹配

next 数组是 KMP 的全部；i 不回溯，只让 j 回退。

03 求模式串 T 的 next 数组

求模式串 T 的 next 数组，下标从 1 开始，next[1] = 0。

时间 O(m)　m = T.length　下标从 1 开始

> **骨架** i = 1, j = 0 → next[1] = 0 → 相等或 j = 0：i、j 同增并写 next[i]；否则 j 回退

```
void get_next(SString T, int next[])
{
    int i = 1, j = 0;
    next[1] = 0;
    while (i < T.length)
    {
        if (j == 0 || T.ch[i] == T.ch[j])
        {
            ++i;
            ++j;                              // 若 p_i = p_j，则 next[j+1] = next[j] + 1
            next[i] = j;
        }
        else
            j = next[j];                      // 否则令 j = next[j]，循环继续
    }
}
```

背诵要点

- 核心 本质：next[j] 表示 j 处失配后，模式串指针回退到的位置，等价于“最长相等前后缀长度 + 1”。
- 易错 循环条件 `i < T.length`：最后一次写入的是 `next[T.length]`。
- 易错 `j == 0` 表示已退到最前，此时 i、j 都要前进。

默写自检：☐ 能写出框架　☐ 能写对边界　☐ 能说出复杂度

04　KMP 算法

主串指针 i 不回溯，利用 next 数组跳过模式串中已匹配的前缀。

时间 O(n+m)　空间 O(m)　i 不回溯

> **骨架**　i = j = 1 → 逐字符比较 → 失配则 j = next[j] → j > T.length 时返回 i - T.length

```
int Index_KMP(SString S, SString T, int next[])
{
    int i = 1, j = 1;
    while (i <= S.length && j <= T.length)
    {
        if (j == 0 || S.ch[i] == T.ch[j])
        {
            ++i;
            ++j;                            // 继续比较后继字符
        }
        else
            j = next[j];                    // 模式串向右移动
    }
    if (j > T.length)                       // 匹配成功
        return i - T.length;
    else
        return 0;
}
```

背诵要点

- 易错　成功判据是 `j > T.length`，返回的是模式串在主串中的起始位置 `i - T.length`。
- 易错　`j == 0` 也要执行 `++i; ++j;`：模式串第一个字符就失配。
- 结论　朴素匹配最坏 O(nm)；KMP 为 O(n+m)，代价是先用 O(m) 求 next。

默写自检：☐ 能写出框架　☐ 能写对边界　☐ 能说出复杂度

三　树与二叉树 · 四种遍历

递归只差 visit 的位置；非递归靠栈，层序靠队列。

05　二叉树的先序遍历（根 → 左 → 右）

递归版三行搞定；非递归版用栈，访问即入栈。

时间 O(n)　空间 O(h)　栈

> **骨架**　递归：visit → 左 → 右；非递归：访问并入栈，一路向左；p 空则弹栈转右

```
void PreOrder1(BiTree T)
{
    if (T != NULL)
    {
        visit(T);                          // 访问根结点
        PreOrder1(T->lchild);              // 递归遍历左子树
        PreOrder1(T->rchild);              // 递归遍历右子树
    }
}

void PreOrder2(BiTree T)
{
    InitStack(S);
    BiTree p = T;                          // 初始化栈 S，p 是遍历指针
    while (p || !IsEmpty(S))               // 栈不空或 p 不空时循环
    {
        if (p)                             // 一路向左
        {
            visit(p);
            Push(S, p);                    // 访问当前结点，并入栈
            p = p->lchild;                 // 左孩子不空，一直向左走
        }
        else                               // 出栈，并转向出栈结点的右子树
        {
            Pop(S, p);                     // 栈顶元素出栈
            p = p->rchild;                 // 向右子树走，p 赋值为当前结点的右孩子
        }                                  // 返回 while 循环继续进入 if-else 语句
    }
}
```

背诵要点

- 核心　三种遍历的递归代码只差 `visit(T)` 放在哪一行。
- 核心　先序非递归：**入栈顺序就是访问顺序**。
- 结论　统一模板：p 非空一路向左；p 为空弹栈转右。

默写自检：☐ 能写出框架　☐ 能写对边界　☐ 能说出复杂度

06　二叉树的中序遍历（左 → 根 → 右）

递归版把 visit 放中间；非递归版一路入栈不访问，出栈时才访问。

时间 O(n)　空间 O(h)　栈

> **骨架**　递归：左 → visit → 右；非递归：一路向左入栈；p 空弹栈访问，再转右

```
void InOrder1(BiTree T)
{
    if (T != NULL)
    {
        InOrder1(T->lchild);                // 递归遍历左子树
        visit(T);                           // 访问根结点
        InOrder1(T->rchild);                // 递归遍历右子树
    }
}

void InOrder2(BiTree T)
{
    InitStack(S);
    BiTree p = T;                           // 初始化栈 S，p 是遍历指针
    while (p || !IsEmpty(S))                // 栈不空或 p 不空时循环
    {
        if (p)                              // 一路向左
        {
            Push(S, p);                     // 当前结点入栈
            p = p->lchild;                  // 左孩子不空，一直向左走
        }
        else                                // 出栈，并转向出栈结点的右子树
        {
            Pop(S, p);
            visit(p);                       // 栈顶元素出栈，访问出栈结点
            p = p->rchild;                  // 向右子树走，p 赋值为当前结点的右孩子
        }                                   // 返回 while 循环继续进入 if-else 语句
    }
}
```

背诵要点

- 核心　中序非递归的关键：**入栈时不访问，出栈时才 visit**。
- 核心　出栈结点访问后，`p = p->rchild`。
- 易错　易错：中序入栈时不要写 visit，出栈时才访问，与先序正好相反。
- 术语　中序序列可用来检查二叉排序树是否有序。

默写自检：☐ 能写出框架　☐ 能写对边界　☐ 能说出复杂度

07　二叉树的后序遍历（左 → 右 → 根）

非递归后序必须记录“最近访问过的结点”，否则会重复走右子树。

时间 O(n)　空间 O(h)　r 记录前驱

> **骨架**　一路向左入栈 → 读栈顶 → 右子树未访问则转右 → 否则弹出访问、r = p、p 置空

```
void PostOrder1(BiTree T)
{
    if (T != NULL)
    {
        PostOrder1(T->lchild);              // 递归遍历左子树
        PostOrder1(T->rchild);              // 递归遍历右子树
        visit(T);                           // 访问根结点
    }
}

void PostOrder2(BiTree T)
{
    InitStack(S);
    BiTree p = T;
    BiTree r = NULL;
    while (p != NULL || !IsEmpty(S))
    {
        if (p != NULL)                      // 走到最左边
        {
            Push(S, p);
            p = p->lchild;
        }
        else                                // 向右
        {
            GetTop(S, p);                   // 读栈顶结点（非出栈）
            if (p->rchild && p->rchild != r)// 若右子树存在，且未被访问过
            {
                p = p->rchild;              // 转向右
            }
            else                            // 否则弹出结点并访问
            {
                Pop(S, p);                  // 将结点弹出
                visit(p->data);             // 访问该结点
                r = p;                      // 记录最近访问的结点
                p = NULL;                   // 结点访问完后，重置 p 指针
            }
        }
    }
}
```

背诵要点

- 易错　判据：`p->rchild && p->rchild != r`（右子树存在且未被访问）。
- 术语　`GetTop` 只读不出栈，`Pop` 才真正弹出。
- 易错　访问后必须 `r = p; p = NULL;`，防止再次向左。
- 结论　后序非递归是三种非递归遍历中最常考的一种。

默写自检：☐ 能写出框架　☐ 能写对边界　☐ 能说出复杂度

08　二叉树的层序遍历

按层从左到右访问，用队列实现。

时间 O(n)　空间 O(w)　队列

> **骨架**　根入队 → 出队访问 → 左孩子入队 → 右孩子入队 → 队空结束

```
void LevelOrder(BiTree T)
{
    LinkQueue Q;                        // 定义辅助队列
    InitQueue(Q);                       // 初始化辅助队列
    BiTree p;
    EnQueue(Q, T);                      // 将根结点入队
    while (!IsEmpty(Q))                 // 队列不空则循环
    {
        DeQueue(Q, p);                  // 队头结点出队
        visit(p);                       // 访问出队结点
        if (p->lchild != NULL)          // 若左孩子不空
            EnQueue(Q, p->lchild);      // 左孩子入队
        if (p->rchild != NULL)          // 若右孩子不空
            EnQueue(Q, p->rchild);      // 右孩子入队
    }
}
```

背诵要点

- 术语　队列先进先出，保证按层从左到右。
- 结论　空间为队列最大长度 O(w)，w 为树的最大宽度。
- 结论　与图 BFS 是同一套“队列 + 标记”的思想。

默写自检：☐ 能写出框架　☐ 能写对边界　☐ 能说出复杂度

四　图 · 遍历与拓扑排序

BFS 用队列、DFS 用递归；非连通图都要外层补调用。

09A　BFS 广度优先搜索（一）：初始化 + 邻接表

先访问所有邻接点，再层层向外扩展。非连通图需对每个未访问顶点补调一次。

邻接表 O(V+E)　空间 O(V)　队列

> **骨架**　visited 初始化 → 每个未访问顶点调用 BFS → 入队即置标记 → 出队 → 未访问邻接点访问并入队

```
bool visited[MaxVertexNum];                 // 标记数组

void BFSTraverse(Graph G)                    // 对图 G 进行广度优先遍历
{
    for (i = 0; i < G.vexnum; i++)           // 标记数组初始化
        visited[i] = false;
    InitQueue(Q);                            // 初始化辅助队列 Q
    for (i = 0; i < G.vexnum; i++)           // 从 0 号顶点开始遍历
        if (!visited[i])                     // 对每个连通分量调用一次 BFS
            BFS(G, i);                       // v_i 未访问过，从 v_i 开始 BFS
}

// 用邻接表实现广度优先搜索
void BFS(ALGraph G, int i)
{
    LinkQueue Q;
    visit(i);                                // 访问初始结点 i
    visited[i] = true;                       // 对 i 做已访问标记
    EnQueue(Q, i);                           // 顶点 i 入队
    while (!IsEmpty(Q))
    {
        DeQueue(Q, v);                       // 队首顶点 v 出队
        for (p = G.vertices[v].firstarc; p; p = p->nextarc)
        {                                    // 检测 v 的所有邻接点
            w = p->adjvex;
            if (visited[w] == false)
            {
                visit(w);                    // w 为 v 的尚未访问的邻接点，访问 w
                visited[w] = true;           // 对 w 做已访问标记
                EnQueue(Q, w);               // 顶点 w 入队
            }
        }
    }
}
```

背诵要点

- 核心　**BFS 在入队时置 visited**，避免同一顶点被重复入队。
- 核心　外层 `BFSTraverse` 负责：初始化标记 + 处理非连通图。
- 结论　邻接表遍历每个顶点的邻接点，总代价 O(V+E)。

默写自检：☐ 能写出框架　☐ 能写对边界　☐ 能说出复杂度

09B　BFS 广度优先搜索（二）：邻接矩阵

存储结构改为邻接矩阵，遍历逻辑不变，只是“找邻接点”的方式变了。

邻接矩阵 $O(V^2)$　空间 $O(V)$　队列

> **骨架**　visit 置标记 → 入队 → 出队 → 用 G.edge[v][w] 扫描所有列 → 未访问则入队

```
// 用邻接矩阵实现广度优先搜索
void BFS(MGraph G, int i)
{
    LinkQueue Q;
    visit(i);                                   // 访问初始结点 i
    visited[i] = true;                          // 对 i 做已访问标记
    EnQueue(Q, i);                              // 顶点 i 入队
    while (!IsEmpty(Q))
    {
        DeQueue(Q, v);                          // 队首顶点 v 出队
        for (w = 0; w < G.vexnum; w++)
        {                                       // 检测 v 的所有邻接点
            if (visited[w] == false && G.edge[v][w] == 1)
            {
                visit(w);                       // w 为 v 的尚未访问的邻接点，访问 w
                visited[w] = true;              // 对 w 做已访问标记
                EnQueue(Q, w);                  // 顶点 w 入队
            }
        }
    }
}
```

背诵要点

- **术语**　邻接矩阵用 `G.edge[v][w] == 1` 判断 v、w 是否相邻。
- **结论**　$O(V^2)$ 与边数无关：稀疏图用邻接表更快。
- **易错**　C 语言没有函数重载：邻接表版与邻接矩阵版需使用不同函数名。

默写自检：☐ 能写出框架　☐ 能写对边界　☐ 能说出复杂度

10　DFS 深度优先搜索

沿一条路走到底再回溯，类比树的先序遍历。

邻接表 O(V+E)　邻接矩阵 O(V^2)　递归栈 O(V)

> **骨架**　visited 初始化 → 每个未访问顶点调用 DFS → visit 并置标记 → 递归所有未访问邻接点

```
// 初始化
bool visited[MaxVertexNum];                    // 标记数组
void DFSTraverse(Graph G)                      // 对图 G 进行深度优先遍历
{
    for (i = 0; i < G.vexnum; i++)             // 标记数组初始化
        visited[i] = false;
    for (i = 0; i < G.vexnum; i++)             // 从 0 号顶点开始遍历
        if (!visited[i])                       // v_i 未访问过，从 v_i 开始 DFS
            DFS(G, i);                         // 对每个连通分量调用一次 DFS
}

// 用邻接表实现深度优先搜索
void DFS(ALGraph G, int i)
{
    visit(i);                                  // 访问初始结点 i
    visited[i] = true;                         // 对 i 做已访问标记
    for (p = G.vertices[i].firstarc; p; p = p->nextarc)
    {                                          // 检测 i 的所有邻接点
        j = p->adjvex;
        if (visited[j] == false)
            DFS(G, j);                         // j 为 i 的尚未访问的邻接点，递归访问 j
    }
}

// 用邻接矩阵实现深度优先搜索
void DFS(MGraph G, int i)
{
    visit(i);                                  // 访问初始结点 i
    visited[i] = true;                         // 对 i 做已访问标记
    for (j = 0; j < G.vexnum; j++)             // 检测 i 的所有邻接点
    {
        if (visited[j] == false && G.edge[i][j] == 1)
            DFS(G, j);                         // j 为 i 的尚未访问的邻接点，递归访问 j
    }
}
```

背诵要点

- 结论　**DFS 在访问时置 visited**；BFS 在入队时置。
- 易错　非连通图同样依靠外层循环补调用，才能遍历完所有分量。
- 术语　邻接矩阵版判断 `G.edge[i][j] == 1`。
- 术语　DFS 是深度优先（Depth First），注意与 BFS 的“广度优先”区分。

默写自检：☐ 能写出框架　☐ 能写对边界　☐ 能说出复杂度

11　拓扑排序

有向无环图中，每次输出一个入度为 0 的顶点，并删除它的所有出边。

邻接表 O(V+E)　空间 O(V)　栈 / 队列

> **骨架**　入度为 0 入栈 → 出栈输出 count++ → 邻接点入度减 1 → 减到 0 入栈 → count 不足则失败

```
bool TopologicalSort(Graph G)
{
    InitStack(S);                              // 初始化栈，存储入度为 0 的顶点
    for (int i = 0; i < G.vexnum; i++)
    {
        if (indegree[i] == 0)
            Push(S, i);                        // 将所有入度为 0 的顶点进栈
    }
    int count = 0;                             // 计数，记录当前已经输出的顶点数
    while (!IsEmpty(S))
    {                                          // 栈不空，则存在入度为 0 的顶点
        Pop(S, i);                             // 栈顶元素出栈
        print[count++] = i;                    // 输出顶点 i
        for (p = G.vertices[i].firstarc; p; p = p->nextarc)
        {   // 将所有 i 指向的顶点的入度减 1，并将入度为 0 的顶点压入栈 S
            v = p->adjvex;
            if (!(--indegree[v]))
                Push(S, v);                    // 入度为 0，则入栈
        }
    }
    if (count < G.vexnum)
        return false;                          // 排序失败
    else
        return true;                           // 排序成功
}
```

背诵要点

- 术语　用栈或用队列都可以，只影响拓扑序列不唯一。
- 易错　`count < G.vexnum` 说明还有顶点未输出，即图中存在环。
- 核心　入度减 1 的写法：`if (!(--indegree[v])) Push(S, v);`
- 结论　先建入度数组，再逐条边“删”。

默写自检：☐ 能写出框架　☐ 能写对边界　☐ 能说出复杂度

五　查找

折半查找三件套：有序、顺序存储、low ⩽ high。

12　折半（二分）查找

在有序顺序表中不断把查找区间折半。

时间 O(log n)　空间 O(1)　有序 + 顺序存储

> **骨架**　low = 0、high = Length-1 → mid = (low+high)/2 → 相等返回 → 小则 high = mid-1 → 大则 low = mid+1

```
int Binary_Search(SSTable L, ElemType key)
{
    // 在有序表中折半查找其关键字等于 key 的数据元素
    // 若找到，则函数值为该元素在表中的位置，否则为 0
    int low = 0;
    int high = L.Tablelength - 1;
    int mid;                                    // 置查找区间初值
    while (low <= high)
    {
        mid = (low + high) / 2;
        if (key == L.elem[mid].key)
            return mid;                         // 找到待查元素
        else if (key < L.elem[mid].key)
            high = mid - 1;                     // 继续在前一子表进行查找
        else
            low = mid + 1;                      // 继续在后一子表进行查找
    }
    return 0;                                   // 表中不存在待查元素
}
```

背诵要点

- 易错　前提：**顺序存储 + 关键字有序**，链表不能折半。
- 易错　循环条件 `low <= high`；high 初值 `Length - 1`。
- 结论　最多比较约 $\lceil \log_2(n+1) \rceil$ 次；返回 0 表示查找失败。

默写自检：☐ 能写出框架　☐ 能写对边界　☐ 能说出复杂度

六　排序

快排划分定枢轴，归并分治再合并。

13　快速排序

选枢轴，一趟划分把小的放左、大的放右，再对两个子表递归。

平均 O(n log n)　最坏 O(n^2)　空间 O(log n)　不稳定

> **骨架**　pivot = data[low] → 右端找小填左坑 → 左端找大填右坑 → 枢轴归位 → 返回 low → 递归左右

```
int Partition(int data[], int low, int high)  // 一趟划分
{
    int pivot = data[low];                    // 将当前表中第一个元素设为枢轴，对表进行划分
    while (low < high)                        // 循环跳出条件
    {
        while (low < high && data[high] >= pivot)  // 将比枢轴小的元素移动到左端
        {
            high--;
        }
        data[low] = data[high];

        while (low < high && data[low] <= pivot)   // 将比枢轴大的元素移动到右端
        {
            low++;
        }
        data[high] = data[low];
    }
    data[low] = pivot;                        // 枢轴元素存放到最终位置
    return low;
}

void QuickSort(int data[], int low, int high)
{
    if (low < high)                           // 递归跳出的条件
    {
        int pivotpos = Partition(data, low, high);
        // Partition() 是划分操作，将表 data[low..high] 划分为满足条件的两个子表
        QuickSort(data, low, pivotpos - 1);
        QuickSort(data, pivotpos + 1, high);
    }
}
```

背诵要点

- 结论　一趟划分至少让枢轴到达最终位置；递归每层都确定一个元素。
- 结论　最坏 O(n^2) 出现在**初始基本有序或逆序**（每次划分极不均匀）。
- 易错　两个内层 while 都必须带 `low < high` 防越界。
- 核心　以第一个元素为枢轴时，先移动 high，再移动 low。

默写自检：☐ 能写出框架　☐ 能写对边界　☐ 能说出复杂度

14　二路归并排序

先二分到单元素，再把相邻的有序段两两合并。

时间 O(n log n)　空间 O(n)　稳定

> **骨架**　复制 A 到 B → i、j 双指针取小写回 A → 处理剩余段 → 递归左右后 Merge

```
ElemType *B = (ElemType *)malloc((n + 1) * sizeof(ElemType)); // 辅助数组 B

void Merge(ElemType A[], int low, int mid, int high)
// 表 A 的两段 A[low..mid] 和 A[mid+1..high] 各自有序，将它们合并成一个有序表
{
    for (int k = low; k <= high; k++)
        B[k] = A[k];                          // 将 A 中所有元素复制到 B 中
    for (i = low, j = mid + 1, k = i; i <= mid && j <= high; k++)
    {
        if (B[i] <= B[j])                     // 比较 B 的左右两段中的元素
            A[k] = B[i++];                    // 将较小值复制到 A 中
        else
            A[k] = B[j++];
    }
    while (i <= mid)
        A[k++] = B[i++];                      // 若第一个表未检测完，复制
    while (j <= high)
        A[k++] = B[j++];                      // 若第二个表未检测完，复制
}

void MergeSort(ElemType A[], int low, int high)
{
    if (low < high)
    {   // 从中间划分两个子序列
        int mid = (low + high) / 2;
        MergeSort(A, low, mid);               // 对左侧子序列进行递归排序
        MergeSort(A, mid + 1, high);          // 对右侧子序列进行递归排序
        Merge(A, low, mid, high);             // 归并
    }
}
```

背诵要点

- 结论　最好、最坏、平均都是 O(n log n)；空间 O(n)；**稳定**。
- 核心　Merge 的第一步是整体复制到 B，之后才双指针比较。
- 易错　k 从 i 开始，而 i 初值为 low；两边剩余段要分别用 while 补齐。
- 术语　分治模板：mid 取中 → 左右递归 → 合并。

默写自检：☐ 能写出框架　☐ 能写对边界　☐ 能说出复杂度

附　录

速查表一　二叉树四种遍历与图遍历

遍历	访问顺序	递归要点	非递归核心	时间 / 空间
先序	根 → 左 → 右	visit 在最前	访问即入栈，一路向左；p 空弹栈转右	O(n) / O(h)
中序	左 → 根 → 右	visit 在中间	入栈不访问，出栈访问，再转右	O(n) / O(h)
后序	左 → 右 → 根	visit 在最后	GetTop 判右子树是否访问过，r 记录前驱	O(n) / O(h)
层序	逐层，从左到右	—	队列：出队访问，左右孩子入队	O(n) / O(w)

对比项	BFS 广度优先	DFS 深度优先
辅助结构	队列	栈 / 递归调用栈
扩展方式	先访问全部邻接点，层层外扩	一条路走到底，再回溯
visited 置位时机	入队时	访问时
邻接表复杂度	O(V+E)	O(V+E)
邻接矩阵复杂度	$O(V^2)$	$O(V^2)$
典型用途	无权图单源最短路径（边数最少）	连通性、拓扑排序、路径搜索

非连通图：BFS / DFS 的外层循环都要遍历所有顶点，对每个未访问顶点补调一次，否则会漏掉连通分量。

速查表二　排序、查找与链表

算法	平均	最好	最坏	空间	稳定性	一句话
快速排序	O(n log n)	O(n log n)	$O(n^2)$	O(log n)	不稳定	枢轴划分，基本有序时最坏
二路归并	O(n log n)	O(n log n)	O(n log n)	O(n)	稳定	二分后合并两个有序段

算法 / 操作	前提	时间	关键点
折半查找	顺序存储且有序	O(log n)	low = 0，high = Length−1，循环 low ⩽ high，失败返回 0
求 next 数组	模式串下标从 1 开始	O(m)	next[1] = 0；相等或 j = 0 时 i、j 同增
KMP	已有 next 数组	O(n+m)	i 不回溯；失配 j = next[j]；成功 j > T.length
拓扑排序	有向图（可判环）	O(V+E)	入度 0 入栈，出栈输出，邻接点入度减 1

对比项	头插法	尾插法
插入位置	头结点之后	表尾结点之后
需要的指针	头指针 L 即可	额外维护尾指针 r
结果顺序	与输入顺序相反	与输入顺序相同
收尾动作	无	r->next = NULL

高频易错清单

1. 两句顺序颠倒 → 链表断裂；正确顺序：先 `s->next = L->next`，再 `L->next = s`。
2. 尾插法 漏写 `r->next = NULL` → 尾结点指针未置空。

3. next 数组下标从 1 开始，`next[1] = 0`，不是从 0 开始。
4. KMP 成功条件是 `j > T.length`，返回 `i - T.length`，**不是 i**。
5. 先序 **入栈即访问**，中序 **出栈才访问**，后序 **右子树访问过才访问**。
6. 后序非递归：`GetTop` 只读不出栈；访问后必须 `r = p; p = NULL;`，否则会重复走左子树。
7. BFS 的 visited **入队时** 置真；DFS 的 visited **访问时** 置真。
8. 非连通图：BFS / DFS 外层 **必须遍历所有顶点**，对每个未访问顶点补调一次。
9. 折半查找 high 初值为 `Length - 1`，循环条件 `low <= high`，失败返回 0。
10. 快排最坏 $O(n^2)$ 出现在 **基本有序 / 逆序**；归并 **稳定** 且额外空间 $O(n)$。
11. 拓扑排序 `count < G.vexnum` 说明 **有环**，排序失败。

三轮默写路线

第一轮 · 认骨架

遮住代码，只看“骨架”一行，口述函数结构与循环。重点：**链表两句插入**、**遍历三行 visit 的位置**。

第二轮 · 扣边界

整段默写，专盯边界：**9999 结束**、**i < T.length**、**low <= high**、**j > T.length**、**r->next = NULL**。

第三轮 · 串模块

按章默写：先链表 → 串 → 树 → 图 → 查找 → 排序。每章写完对照“高频易错清单”自查。

默写自检表

编号	内容	能写框架	能写边界	能说复杂度
01	头插法建立单链表	☐	☐	☐
02	尾插法建立单链表	☐	☐	☐
03	求模式串 T 的 next 数组	☐	☐	☐
04	KMP 算法	☐	☐	☐
05	二叉树的先序遍历	☐	☐	☐
06	二叉树的中序遍历	☐	☐	☐
07	二叉树的后序遍历	☐	☐	☐
08	二叉树的层序遍历	☐	☐	☐
09A	BFS（初始化 + 邻接表）	☐	☐	☐
09B	BFS（邻接矩阵）	☐	☐	☐
10	DFS 深度优先搜索	☐	☐	☐
11	拓扑排序	☐	☐	☐
12	折半（二分）查找	☐	☐	☐
13	快速排序	☐	☐	☐
14	二路归并排序	☐	☐	☐

打印建议：A4 单面或双面均可；四色均为浅底纹，彩色打印清晰，黑白打印靠加粗与深浅仍可分辨。